

FTS

Формальная семантика, проверяемые сертификаты и патентная техническая основа

Конфиденциальный исследовательский препринт

Документ объединяет формальную модель, доказательства метасвойств, ограничения результата, фальсифицируемый тестовый протокол и техническое раскрытие для последующего патентного исследования.

НЕ ПУБЛИКОВАТЬ ДО ФИКСАЦИИ ПАТЕНТНОЙ СТРАТЕГИИ И ПРИОРИТЕТА

Редакция: 2026-08-01

Ядро сертификата: fts-proof/1

Статус: конфиденциальный исследовательский препринт. Не публиковать до определения патентной стратегии и даты приоритета.

Аннотация

Предлагается Formal Type Surface (FTS) - формальный язык, близкий по синтаксической форме к TypeScript и предназначенный для машинного описания структур данных, типизированных морфизмов, утверждений и проверяемых свидетельств. В отличие от декларативных схем, результат обработки FTS включает доказательственный сертификат: конечную последовательность локально проверяемых правил вывода, связанную каноническими криптографическими дайджестами с исходным документом и контекстом данных. Определены синтаксис формального ядра, отношение разрешения JSON-пути, система типов, правила `witness` и `apply`, алгоритмы построения и независимой проверки сертификата. Доказаны детерминированность разрешения пути, единственность типа вывода, сохранение типов, завершаемость проверки и относительная корректность сертификата. Ограничение результата сформулировано явно: истинность объявленных морфизмов является предпосылкой, тогда как фактическое свидетельство проверяется по предоставленному контексту.

Ключевые слова: формальные методы, типы как утверждения, proof-carrying data, проверка доказательств, TypeScript, категориальная композиция, JSON, SHA-256, искусственные агенты.

1. Постановка задачи

Современные программные и агентные системы обмениваются структурированными данными и декларациями правил, однако наличие корректного JSON или статически корректного интерфейса не доказывает выполнение утверждения над конкретным состоянием данных. Дополнительная проблема возникает при использовании генеративных агентов: текст правила, программа проверки и объяснение результата могут быть созданы разными компонентами и расходиться.

Требуется обеспечить одновременно:

1. человекочитаемую поверхность, близкую к распространённой системе типов;
2. каноническое машинное представление;
3. проверку фактических свидетельств над JSON-контекстом;
4. типизированную композицию выводов;
5. компактный сертификат, проверяемый независимо от производителя;
6. детерминированный интерфейс для программ и ИИ-агентов.

FTS решает эту задачу разделением трёх уровней: `surface syntax`, канонический документ и `proof certificate`.

2. Связанные основания

Интерпретация «утверждения как типы, доказательства как программы» восходит к работе У. Говарда [1]. Представление правил и доказательств через типизированные суждения развито в Edinburgh Logical Framework, где проверка доказательства сводится к проверке типов [2]. Архитектурный принцип отделения производителя доказательства от небольшого проверяющего компонента соответствует proof-carrying code Дж. Некулы [3]. Категории и функторы используются в исходном смысле структур и сохраняющих композицию отображений [4]. Для воспроизводимых дайджестов JSON применяется канонизация по принципам JCS [5].

FTS не претендует на замену универсального proof assistant. Его предмет - ограниченный, разрешимый и пригодный для интеграции слой доказательств над типизированными JSON-данными.

3. Формальная модель

3.1. Базовые множества

Пусть:

- Id - множество допустимых идентификаторов;
- Type - множество номинальных типов;
- Val - множество I-JSON значений;
- Path - конечные последовательности сегментов JSON-пути;
- Ctx - множество JSON-контекстов.

Сегмент пути имеет одну из форм:

$$s ::= \text{key}(k) \mid \text{index}(n) \mid \text{select}(q)$$

где $k \in \text{String}$, $n \in \mathbb{N}$, а q - конечное отображение строковых ключей в скалярные JSON-значения.

3.2. Сигнатура документа

Документ задаёт сигнатуру

$$\text{Sigma} = (C, \text{Delta}, M)$$

где C - имя категории, $\text{Delta} : \text{Structure} \times \text{Field} \rightarrow \text{Type}$ - частичное отображение полей в типы, $M : \text{Functor} \rightarrow \text{Type} \times \text{Type} \times \text{Law}$ - частичное отображение функторов в домен, кодомен и объявленный закон.

Выражения доказательного ядра:

$$e ::= \text{witness}(S.f, p, v) \mid \text{apply}(m, e) \mid \text{compose}([m_1, \dots, m_n], e)$$

compose является синтаксическим сахаром последовательного apply слева направо.

3.3. Разрешение пути

Суждение

$$\text{Gamma} \vdash p \Rightarrow v$$

означает: путь p в контексте Gamma детерминированно разрешается в значение v .

Правила:

$$\begin{array}{c} \frac{}{\text{Gamma} \vdash [] \Rightarrow \text{Gamma}} \text{ROOT} \\ \\ \frac{\text{Gamma} \vdash p \Rightarrow o \quad o[k] = v}{\text{Gamma} \vdash p \cdot \text{key}(k) \Rightarrow v} \text{KEY} \\ \\ \frac{\text{Gamma} \vdash p \Rightarrow a \quad 0 \leq n < |a|}{\text{Gamma} \vdash p \cdot \text{index}(n) \Rightarrow a[n]} \text{INDEX} \\ \\ \frac{\text{Gamma} \vdash p \Rightarrow a \quad \text{unique } x \text{ in } a : x \sqsubseteq q}{\text{Gamma} \vdash p \cdot \text{select}(q) \Rightarrow x} \text{SELECT} \end{array}$$

Реализация требует ровно одного совпавшего элемента массива. Отсутствующий или неоднозначный `selector` отклоняется, поэтому исполняемое правило буквально совпадает с `SELECT`.

3.4. Система вывода

Основное суждение:

$$\text{Sigma}; \text{Gamma} \vdash e : \tau \mid \pi$$

где π - последовательность шагов сертификата.

Правило фактического свидетельства:

$$\frac{\text{Delta}(S, f) = \tau \quad \text{Gamma} \vdash p \Rightarrow v}{\text{Sigma}; \text{Gamma} \vdash \text{witness}(S, f, p, v) : \tau \mid [w]} \text{ WITNESS}$$

Шаг w содержит структуру, поле, тип, путь, ожидаемое и фактическое значения, признак проверки и digest evidence.

Правило применения морфизма:

$$\frac{\text{Sigma}; \text{Gamma} \vdash e : \tau \mid \pi \quad M(m) = (\tau, \tau', \text{law})}{\text{Sigma}; \text{Gamma} \vdash \text{apply}(m, e) : \tau' \mid \pi \cdot [a]} \text{ APPLY}$$

Шаг a содержит имя функтора, домен, кодомен и закон, учитываемый как явная предпосылка.

Правило композиции выводится повторным применением APPLY:

$$\frac{\begin{array}{l} \text{Sigma}; \text{Gamma} \vdash e : \tau_0 \mid \pi_0 \\ M(m_i) = (\tau_{i-1}, \tau_i, \text{law}_i), i = 1..n \end{array}}{\text{Sigma}; \text{Gamma} \vdash \text{compose}([m_1, \dots, m_n], e) : \tau_n \mid \pi_0 \cdot [a_1, \dots, a_n]} \text{ COMPOSE}$$

4. Доказательственный сертификат

Сертификат P имеет поля:

```
P = (version, category, status,
      Hdoc, Hctx?, assumptions,
      steps, conclusion, Hcert)
```

$Hdoc$ и $Hctx$ - SHA-256 дайджесты канонических JSON-представлений документа и контекста. $Hcert$ вычисляется над всеми полями сертификата, кроме самого $Hcert$. Канонизация рекурсивно сортирует ключи по кодовым единицам UTF-16, не изменяет порядок массивов и отвергает нечисловые значения, бесконечности и отрицательный ноль.

Статус `verified` присваивается только при фактической проверке каждого `witness`. При отсутствии контекста или полного пути статус равен `symbolic`; такой объект является схемой вывода, но не строгим фактическим доказательством.

Независимый verifier:

1. нормализует и валидирует FTS-документ;
2. заново строит вывод из документа и контекста;
3. заново вычисляет $Hdoc$, $Hctx$, `evidence digest` и ожидаемый $Hcert$;
4. вычисляет `digest` переданного `payload` сертификата;
5. принимает сертификат, только если оба `digest` совпадают с заявленным;
6. для строгого режима дополнительно требует `status = verified`.

5. Основные свойства

Теорема 1. Детерминированность разрешения пути

Если $\text{Gamma} \vdash p \Rightarrow v_1$ и $\text{Gamma} \vdash p \Rightarrow v_2$, то $v_1 = v_2$, при условии функциональности `select`.

Доказательство. Индукция по длине p . Для пустого пути оба результата равны Γ . В индукционном переходе префикс пути по предположению индукции разрешается в единственный объект или массив. Доступ по ключу и индексу является функциональным. При `select` функциональность обеспечивается условием единственности совпадающего элемента. Следовательно, последний сегмент также имеет единственный результат. [QED]

Теорема 2. Единственность типа вывода

Если $\Sigma; \Gamma \vdash e : \tau_1 \mid \pi_1$ и $\Sigma; \Gamma \vdash e : \tau_2 \mid \pi_2$, то $\tau_1 = \tau_2$.

Доказательство. Структурная индукция по e . Для `witness` тип однозначно задаётся частичной функцией $\Delta(S, f)$. Для `apply` по предположению индукции тип аргумента единственен, а $M(m)$ задаёт единственный кодомен. `Compose` сводится к конечной последовательности `apply`. [QED]

Теорема 3. Сохранение типов при композиции

Если $\Sigma; \Gamma \vdash e : \tau_0 \mid \pi_0$ и для каждого i выполнено $M(m_i) = (\tau_{i-1}, \tau_i, \text{law}_i)$, то $\Sigma; \Gamma \vdash \text{compose}([m_1, \dots, m_n], e) : \tau_n$.

Доказательство. Индукция по n . При $n=0$ заключение совпадает с исходным суждением. Для $n+1$ по предположению индукции первые n функторов дают выражение типа τ_n ; правило `APPLY` для $m_{n+1} : \tau_n \rightarrow \tau_{n+1}$ даёт требуемый тип. [QED]

Теорема 4. Завершаемость проверки

Для конечного FTS-документа, конечного JSON-контекста и конечного сертификата алгоритмы `certify` и `verify` завершаются.

Доказательство. Парсер проходит конечную последовательность токенов. Валидация проходит конечные списки деклараций. Разрешение пути выполняет не более $|p|$ шагов; `selector` просматривает конечный массив. Вывод рекурсирует по конечному синтаксическому дереву, а `compose` - по конечному списку функторов. Канонизация рекурсирует по конечному JSON-дереву. Ни один этап не содержит неограниченного поиска или рекурсивного правила, увеличивающего вход. [QED]

Теорема 5. Относительная корректность

Пусть интерпретация $[[\tau]]$ сопоставляет типу множество истинных свидетельств. Если:

1. каждый WITNESS разрешён в Γ и его значение принадлежит $[[\tau]]$;
2. каждый объявленный функтор $m : \tau \rightarrow \tau'$ семантически сохраняет истинность, то есть переводит элементы $[[\tau]]$ в $[[\tau']]$;
3. `verifier` принял сертификат со статусом `verified`,

то заключение сертификата принадлежит интерпретации его типа.

Доказательство. Индукция по списку шагов. База следует из условия 1 и правила WITNESS. Индукционный переход следует из условия 2 и правила APPLY. Для `compose` применяется переход последовательно. Проверка сертификата гарантирует, что рассматриваемая последовательность совпадает с детерминированно восстановленной последовательностью документа и контекста. [QED]

Теорема является относительной: FTS проверяет данные и корректность применения объявленных морфизмов, но не доказывает внешнюю аксиому `law` из ничего. Это ограничение фиксируется в `assumptions`.

Теорема 6. Обнаружение модификации сертификата

При предположении стойкости SHA-256 к нахождению второго прообраза изменение канонического `payload` сертификата без соответствующего изменения `hcert` обнаруживается `verifier`ом.

Доказательство. Verifier вычисляет digest фактически переданного payload и сравнивает его с H_{cert} . Необнаруженное изменение требует другого payload с тем же SHA-256 digest, то есть второго прообраза. [QED]

6. Алгоритмическая сложность

Пусть N - размер документа, G - размер контекста, P - длина пути, K - число шагов вывода. Парсинг и структурная валидация выполняются за $O(N)$, без учёта таблиц дубликатов. Разрешение пути имеет сложность $O(P + \sum |a_i|)$ для selector-сегментов. Проверка типов цепочки - $O(K)$ при индексированных функторах и $O(KM)$ в текущей линейной реализации поиска. Канонизация и хеширование требуют $O((N+G) \log B)$ из-за сортировки ключей объектов, где B - максимальное число ключей объекта.

7. Фальсифицируемые свойства и тестовый протокол

Система должна отклонять:

- несовпадение expected/actual witness;
- отсутствующий путь;
- применение функтора к неверному доменному типу;
- неизвестное поле, структуру или функтор;
- изменение conclusion при сохранённом старом digest;
- изменение документа или контекста после выдачи сертификата;
- сертификат неизвестной версии;
- strict verify символического сертификата.

Положительные тесты должны подтверждать воспроизводимость digest при перестановке ключей входного JSON и одинаковый вывод CLI, library API и MCP.

8. Научная новизна как проверяемая гипотеза

Исследовательская гипотеза состоит не в изобретении Curry-Howard correspondence или категориальной композиции. Проверяемый вклад FTS формулируется как интеграция:

1. TypeScript-подобной поверхности предметных типов;
2. JSON evidence path с selector-сегментами;
3. конечного типизированного proof certificate;
4. независимого воспроизводимого verifier;
5. единого интерфейса для человека, утилиты и ИИ-агента.

Заявление о мировой новизне требует отдельного патентного и библиографического поиска. До такого поиска корректно говорить о предлагаемой архитектуре и доказанных свойствах конкретной формальной модели.

9. Ограничения и направления развития

- Selector намеренно требует единственного совпадения; прикладные адаптеры должны обеспечивать устойчивые идентификаторы элементов.
- Законы функторов являются доверенными предпосылками. Следующая версия может допускать ссылки на отдельные сертификаты законов.

- SHA-256 связывает артефакты, но не устанавливает авторство; для этого нужна цифровая подпись сертификата.
- Номинальное равенство типов не учитывает параметрические и зависимые типы.
- Нужны дифференциальные тесты канонизации на полном наборе RFC 8785.

10. Заключение

FTS задаёт малое разрешимое ядро, в котором проверка факта и проверка типизированной композиции объединены в воспроизводимый proof certificate. Основные метасвойства доказываются индукцией по пути и структуре вывода. Практическая ценность заключается в возможности отделить производителя правила или ИИ-агента от небольшого независимого verifier'a и сохранить аудируемую связь между моделью, данными, предпосылками и заключением.

Литература

1. Howard W. A. The Formulae-as-Types Notion of Construction // To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism. Academic Press, 1980.
2. Harper R., Honsell F., Plotkin G. A Framework for Defining Logics // Journal of the ACM. 1993. Vol. 40, No. 1. P. 143-184. <https://era.ed.ac.uk/items/d9cebf2b-52bd-4532-b848-7d0402f32cf5/full>
3. Necula G. C. Proof-Carrying Code // POPL '97. P. 106-119. <https://doi.org/10.1145/263699.263712>
4. Eilenberg S., Mac Lane S. General Theory of Natural Equivalences // Transactions of the AMS. 1945. Vol. 58. P. 231-294. <https://www.ams.org/journals/tran/1945-058-00/S0002-9947-1945-0013131-6/>
5. Rundgren A., Jordan B., Erdtman S. JSON Canonicalization Scheme (JCS). RFC 8785, 2020. <https://www.rfc-editor.org/rfc/rfc8785.html>
6. Wadler P. Propositions as Types // Communications of the ACM. 2015. Vol. 58, No. 12. P. 75-84. <https://homepages.inf.ed.ac.uk/wadler/papers/propositions-as-types/stanford.pdf>

Приложение А. Патентная техническая основа

Статус: конфиденциально. Черновик технического материала, а не юридическое заключение.

1. Режим раскрытия

Репозиторий и этот документ следует сохранять закрытыми до выбора стратегии охраны и фиксации приоритета. Публичный препринт, доклад, демонстрация или открытый исходный код могут стать раскрытием. Статья 1350 ГК РФ содержит шестимесячную льготу для раскрытия автором, однако обязанность доказать обстоятельства раскрытия лежит на заявителе. Практически безопасная последовательность: поиск уровня техники, консультация патентного поверенного, подача заявки, затем публикация.

2. Область техники

Решение относится к машинной проверке результатов обработки структурированных данных, управлению выполнением программных действий и контролю инструментальных вызовов автоматизированных и агентных систем.

3. Техническая проблема

Существующие декларативные схемы подтверждают форму данных, но сами по себе не связывают:

- исходную формальную модель;
- конкретное состояние данных;
- последовательность типизированных преобразований;
- заключение;
- решение о разрешении машинного действия.

Текстовое объяснение ИИ-агента также не является независимо проверяемым доказательством. Полное повторное выполнение исходного производителя может быть дорогим, недетерминированным или недоступным доверяющей стороне.

4. Предлагаемый технический результат

Предлагаемая система снижает риск принятия изменённого, несогласованного или типово некорректного результата за счёт отдельного детерминированного verifier. Сертификат криптографически связан с нормализованной моделью и контекстом, а прикладное действие разрешается только после повторного построения типовой цепочки и проверки фактического witness.

Измеримые эффекты:

- обнаружение изменений модели, контекста, шагов или заключения;
- блокирование типово несовместимых цепочек;
- воспроизводимая проверка независимой реализацией;
- уменьшение доверенной вычислительной базы до нормализатора, правил ядра, канонизатора и verifier;
- возможность проверять результат агента до вызова инструмента с внешним эффектом.

5. Существенная последовательность способа

1. Получить текстовую типизированную модель FTS и структурированный контекст данных.

2. Преобразовать модель в канонический документ со структурами, типами полей, направленными преобразованиями и целевым утверждением.
3. Проверить ссылочную и типовую целостность документа.
4. Разрешить путь свидетельства в контексте; selector обязан иметь ровно одно совпадение.
5. Сравнить фактическое и ожидаемое значения.
6. Сформировать начальный типизированный шаг *witness*.
7. Последовательно применять преобразования только при совпадении текущего типа с доменом следующего преобразования.
8. Сформировать сертификат со шагами, заключением и явными предпосылками.
9. Канонизировать документ, контекст, *evidence* и *payload* сертификата.
10. Вычислить криптографические дайджесты канонических представлений.
11. Передать сертификат независимому проверяющему модулю.
12. Повторно построить ожидаемую последовательность вывода и дайджесты.
13. Сравнить переданные и вычисленные значения.
14. Разрешить машинное действие только при строгом статусе *verified*.

6. Существенные признаки системы

Система содержит модуль синтаксического анализа FTS, нормализатор, семантический валидатор, модуль доступа к JSON-контексту, построитель сертификата, канонизатор, модуль дайджестов, независимый *verifier*, интерфейс к исполнителю или ИИ-агенту и журнал аудита.

7. Черновой независимый пункт способа

Способ машинной верификации результата обработки структурированных данных, выполняемый по меньшей мере одним процессором, включающий получение формальной модели, содержащей типы полей, направленные типизированные преобразования и утверждение; получение контекста структурированных данных; разрешение заданного в утверждении пути в указанном контексте с получением фактического значения; формирование шага свидетельства при совпадении фактического значения с ожидаемым; формирование последовательности шагов преобразования, причём каждый последующий шаг формируют только при совпадении доменного типа преобразования с типом результата предшествующего шага; формирование доказательственного сертификата, содержащего указанную последовательность, заключение и криптографические дайджесты канонических представлений модели и контекста; передачу сертификата проверяющему модулю, отличному от модуля формирования; повторное построение проверяющим модулем последовательности шагов и вычисление дайджеста сертификата; и разрешение машинно-исполняемого действия только при совпадении повторно построенной последовательности и вычисленных дайджестов с данными сертификата.

Формулировка требует патентного поиска, проверки патентным поверенным и привязки к конкретному техническому контуру.

8. Возможные зависимые признаки

- selector однозначно определяет элемент массива;
- контекст представлен I-JSON;
- канонизация совместима с RFC 8785;

- сертификат подписывается электронной подписью;
- verifier исполняется в изолированной доверенной среде;
- сертификат сопровождает запрос агента к инструменту;
- действие блокируется при статусе `symbolic`;
- законы преобразований ссылаются на сертификаты нижнего уровня;
- журнал образует цепочку дайджестов состояний.

9. Границы возможной новизны

Не следует заявлять как новые сами по себе `propositions-as-types`, категории и композицию, статическую типизацию, JSON Schema, криптографический hash, `proof-carrying code`, MCP или вызов инструментов агентами. Потенциально охраноспособной может быть конкретная совокупность признаков и достигаемый технический результат. Вывод о мировой новизне и изобретательском уровне допустим только после поиска патентов и заявок, включая релевантные подклассы G06F.

10. Экспериментальная программа

Следует измерить:

1. время полного повторного анализа и время проверки сертификата;
2. долю обнаруженных модификаций документа, контекста, шага и заключения;
3. долю заблокированных типово несовместимых agent tool calls;
4. размер сертификата относительно контекста;
5. воспроизводимость дайджеста независимыми реализациями;
6. накладные расходы verifier на наборах от 10^2 до 10^6 элементов.

Контрольные группы: JSON Schema без сертификата, полное повторное выполнение, текстовое объяснение агента без verifier и цифровая подпись результата без типизированной цепочки.

11. Материалы до подачи

- отчёт о патентном поиске;
- диаграмма аппаратно-программного контура;
- листинг reference implementation;
- корректные и отклонённые сертификаты;
- benchmark verifier;
- формальное описание алгоритма;
- сведения об авторах и правообладателе;
- решение о регистрации программы для ЭВМ;
- рецензия патентного поверенного на формулу.

12. Правовая рамка, проверенная на 1 августа 2026 года

- [ГК РФ, часть IV, статья 1350](#): условия патентоспособности, исключения и шестимесячная льгота по раскрытию.
- [Приказ Минэкономразвития России от 21.02.2023 № 107](#): правила и требования к заявке на изобретение.

- [Приказ Минэкономразвития России от 09.02.2026 № 94](#): актуальные изменения к приказу № 107, вступившие в силу 10.03.2026.
- [Государственная регистрация программы для ЭВМ](#): отдельный режим охраны исходного кода, не заменяющий патент на техническое решение.

Перед подачей правовой раздел необходимо проверить повторно по актуальным официальным текстам.